Práctica 2 - Redes Neuronales Convolucionales

Rafael Pastor Vargas / **Manuel Castillo-Cara** / Jorge Pérez / José Manuel Cuadra
Curso académico: 2025/2026

16 de febrero de 2026

Introducción

En este documento se propone la segunda actividad de la asignatura. Esta actividad corresponde a los contenidos de las Redes Neuronales Convolucionales de la asignatura y al despliegue de modelos en la nube (Servicios y proveedores de Deep Learning en la nube), y antes de su realización se asume que el alumnado ha leído el material propuesto en la guía de studiao.

El objetivo de esta actividad es que el alumnado ponga en práctica los contenidos teóricos relativos a las redes neuronales y el aprendizaje profundo para asentar su conocimiento de estas técnicas. Una vez desarrolladas las habilidades sibre uso de redes neuronales, se pretende que el estudiante sea capaz de desplegar el modelo entrenado en un entorno de ejecución, en este caso, en la nube. Es importante dejar claro que, como ocurre en el resto de las actividades evaluables de esta asignatura, lo importante no es tanto resolver un problema propuesto por el equipo docente como demostrar que se han comprendido e interiorizado los diversos aspectos clave de las técnicas que se ponen en juego, y su impacto en la implementación y resultados.

De esta manera, la práctica se divide en dos partes, una primera centrada en la definición y entrenamiento de redes neuronales, mas una segunda orientada a la parte más de ingeniería en la puesta en producción de servicios de IA.

Importante: Esta PEC se divide en **dos apartados independientes**:

1. **Apartado 1:** Redes Neuronales Convolucionales (CNN).
2. **Apartado 2:** Despliegue del modelo en la nube.

Es fundamental que leáis con atención la sección **"Entregables"**, ya que cada apartado tiene requerimientos específicos y se exige la entrega de materiales diferenciados.

Apartado 1: Redes Neuronales Convolucionales

Aprendizaje profundo

Para el desarrollo de esta actividad, el equipo docente recomienda el uso de la librería *Keras* [5] sobre *Python*, debido a su facilidad de uso y amplia adopción en la comunidad científica. No obstante, su uso no es obligatorio: el alumnado podrá optar por otras alternativas, como *PyTorch* u otros lenguajes de programación, siempre que se justifique la elección realizada.

En caso de no contar con experiencia previa, se aconseja comenzar revisando y completando las prácticas de programación correspondientes al bloque de Aprendizaje Profundo, que están orientadas al uso de *Keras*. De forma complementaria, puede resultar útil seguir el tutorial de clasificación básica con *Keras* (disponible tanto para Python [1] como para `R` [2]), y reproducir sus pasos empleando los datos proporcionados por el equipo docente.

Asimismo, si se decide utilizar un software alternativo de aprendizaje profundo, se recomienda compartir esta elección con el resto de compañeros y con el equipo docente a través del foro de

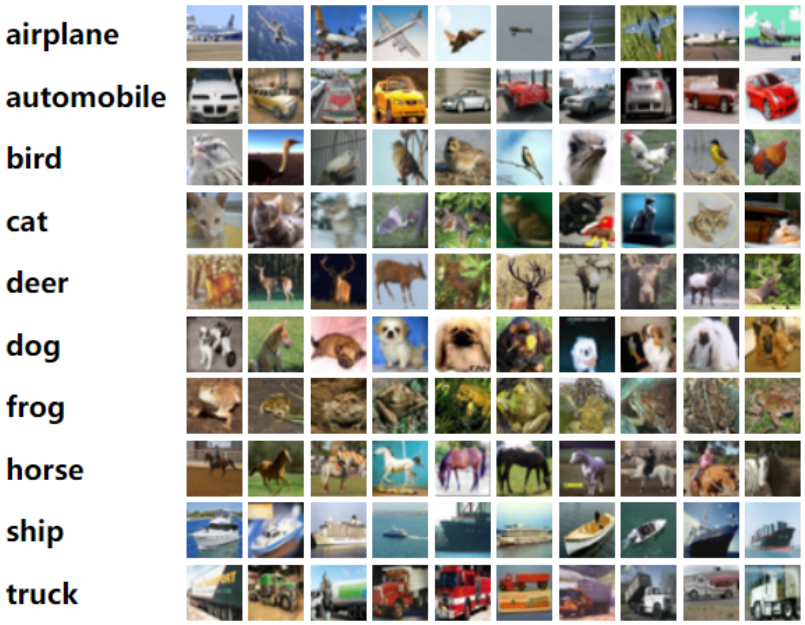


Figura 1: Ejemplos de imágenes del dataset CIFAR-10.

la asignatura, indicando los motivos de dicha decisión y las ventajas potenciales que dicho entorno puede ofrecer en relación con la tarea planteada.

Dataset de trabajo en la PEC

En esta PEC se trabajará con el conjunto de datos **CIFAR-10**, ampliamente utilizado en tareas de reconocimiento de objetos en imágenes. Este dataset contiene imágenes etiquetadas en 10 clases distintas y constituye un estándar de referencia en la investigación de visión por computador.

El nombre CIFAR proviene del *Canadian Institute for Advanced Research*, donde Alex Krizhevsky, Vinod Nair y Geoffrey Hinton (uno de los pioneros en el campo de la Inteligencia Artificial) recopilaron este subconjunto a partir del conjunto de datos *80 Million Tiny Images*. La Figura 1 muestra ejemplos de imágenes pertenecientes a las diferentes clases que lo componen.

Las características principales de CIFAR-10 son las siguientes:

- Consta de un total de 60.000 imágenes a color.
- Cada imagen tiene dimensiones de 32×32 píxeles con 3 canales (RGB).
- 50.000 imágenes están destinadas al entrenamiento.
- 10.000 imágenes se reservan para la prueba.
- El conjunto incluye 10 clases, con 6.000 imágenes por clase.
- Las clases son mutuamente excluyentes: cada imagen pertenece únicamente a una categoría.

Las 10 clases de CIFAR-10 son:

- `airplane` (avión),
- `automobile` (automóvil),
- `bird` (ave),

- `cat` (gato),
- `deer` (ciervo),
- `dog` (perro),
- `frog` (rana),
- `horse` (caballo),
- `ship` (barco),
- `truck` (camión).

Para ampliar información, se recomienda consultar:

- Documentación oficial: https://www.cs.toronto.edu/~kriz/cifar.html
- Implementación en Keras: https://keras.io/api/datasets/cifar10/

Particiones y evaluación del modelo

Para el desarrollo de esta práctica, es obligatorio dividir el conjunto de datos en tres subconjuntos bien diferenciados: **entrenamiento (train)**, **validación (val)** y **test**. Esta separación es esencial para garantizar un entrenamiento adecuado del modelo y una evaluación objetiva de su rendimiento.

- **Curvas de aprendizaje**: deberán construirse empleando las métricas obtenidas sobre el conjunto de **validación**, ya que es este subconjunto el que se utiliza durante el entrenamiento para ajustar los hiperparámetros y monitorizar la evolución del modelo.
- **Resultados finales**: las métricas de rendimiento (e.g., *accuracy*, *precision*, *recall*, *F1-score*) deberán calcularse y reportarse únicamente sobre el conjunto de **test**, que debe permanecer completamente aislado durante todo el proceso de entrenamiento. Esto garantiza que las conclusiones extraídas sean sólidas y representativas de la capacidad de generalización del modelo.

Este protocolo de evaluación resulta fundamental para evitar el *overfitting* y asegurar que las conclusiones acerca del rendimiento de los modelos sean **fiables, reproducibles y generalizables** a nuevos conjuntos de datos.

Especificación de la tarea

Análisis y preprocesamiento de datos

El primer paso en el desarrollo de esta práctica consiste en realizar un **análisis exploratorio** y un **preprocesamiento adecuado** del conjunto de datos. Este trabajo inicial resulta fundamental para garantizar que los modelos posteriores puedan entrenarse de manera eficiente y obtener un rendimiento fiable.

En esta fase se recomienda examinar aspectos clave como:

- **Balance de clases**: verificar si las categorías están representadas de forma equitativa o si existen desbalances que puedan sesgar el aprendizaje del modelo.
- **Técnicas de *data augmentation***: aplicar estrategias de aumento de datos (rotaciones, traslaciones, volteos, cambios de escala, entre otros) que permitan incrementar la diversidad del conjunto de entrenamiento y mejorar la capacidad de generalización del modelo.
- **Redimensionamiento de imágenes**: adaptar las imágenes al tamaño requerido por la arquitectura seleccionada, asegurando coherencia en el formato de entrada.

- **Normalización**: escalar los valores de los píxeles (por ejemplo, en el rango [0,1] o mediante estandarización) para facilitar el proceso de optimización y acelerar la convergencia del entrenamiento.

IMPORTANTE: No se evaluará únicamente la aplicación mecánica de estas técnicas, sino la **justificación razonada** de cada decisión tomada. Se espera que el alumnado argumente por qué ha aplicado (o descartado) cada técnica y cuál ha sido su impacto en la calidad de los datos y en la preparación para el modelado.

Modelado

A partir del conjunto de imágenes CIFAR-10 se propone la construcción de un sistema de **clasificación automática** de las 10 clases que lo componen. El objetivo no es únicamente implementar los modelos, sino también **analizar críticamente su comportamiento** y extraer conclusiones fundamentadas.

Se deberán construir, como mínimo, los siguientes modelos:

1. **Modelo 1: CNN de línea base.** Construir una red neuronal convolucional (*CNN*) desde cero que sirva como modelo de referencia inicial. El diseño de la arquitectura (número y tipo de capas) queda a criterio del estudiante, pero debe justificarse empíricamente en función de la convergencia lograda y las métricas obtenidas. Este será el **resultado de línea base**.

2. **Modelo 2: Aprendizaje por transferencia.** Seleccionar un modelo preentrenado disponible en *Keras Applications*[1] (u otra librería si se utiliza un software alternativo) y adaptarlo al dataset. En este enfoque, el modelo preentrenado se utilizará como **extractor de características fijo**: las capas finales de clasificación se sustituyen por nuevas capas específicas para la tarea de CIFAR-10, mientras que los pesos del modelo original permanecen congelados.

3. **Modelo 3: Ajuste fino (*Fine-tuning*).** A partir del modelo anterior, se descongelarán selectivamente algunas capas del modelo preentrenado para permitir su actualización con el nuevo conjunto de datos. Este proceso de ajuste fino permite al modelo adaptar su representación interna, aprendiendo características más específicas del dominio de CIFAR-10, al tiempo que conserva el conocimiento general adquirido en el dataset original de entrenamiento.

Nota importante: Mientras que el aprendizaje por transferencia implica mantener los pesos del modelo preentrenado congelados y entrenar únicamente las capas añadidas, el ajuste fino va un paso más allá, permitiendo actualizar parcialmente los pesos originales para optimizar el rendimiento en la nueva tarea. Se espera que el alumnado discuta las diferencias entre ambos enfoques y justifique sus elecciones en términos de resultados experimentales y métricas de evaluación.

Explicabilidad

Además de los modelos anteriores, se debe implementar **GradCAM** (*Gradient-weighted Class Activation Mapping*) como técnica de interpretabilidad para analizar las decisiones de las CNN. GradCAM permite visualizar qué regiones de una imagen influyen más en la predicción de una clase específica, lo que resulta especialmente útil para comprender cómo la red toma decisiones y para identificar posibles errores o sesgos en el modelo.

Para este apartado se podrá utilizar el modelo CNN que se considere más conveniente, siempre y cuando ofrezca un rendimiento cuantitativo y cualitativo adecuado.

Se sugieren los siguientes recursos de referencia (aunque el alumnado puede seleccionar otros equivalentes):

- GradCAM en Keras: https://keras.io/examples/vision/grad_cam/

[1] https://keras.io/api/applications/

- GradCAM y variantes en PyTorch: https://github.com/jacobgil/pytorch-grad-cam

La tarea se estructura en dos bloques principales:

1. **Visualización de GradCAM:**
   a) Generar mapas de activación utilizando GradCAM y representarlos con `Matplotlib` (u otra librería de visualización).
   b) Superponer los mapas de activación sobre las imágenes originales para identificar qué regiones han sido más relevantes en la predicción.
   c) Se recomienda trabajar con al menos dos imágenes por clase (20 en total) para garantizar un análisis representativo.
2. **Análisis crítico:** Este es el aspecto más relevante de la tarea, ya que no basta con mostrar los mapas, sino que se debe realizar un análisis detallado y reflexivo:
   a) **Discusión de los mapas de activación:** ¿Qué áreas de la imagen son más influyentes en la predicción? ¿Qué características (bordes, texturas, formas) parecen estar captando los filtros?
   b) **Comparación entre clases e instancias:** Contrastar los mapas obtenidos para imágenes de la misma clase y entre diferentes clases. Identificar patrones comunes o diferencias significativas.
   c) **Interpretación de los resultados:** Reflexionar sobre qué conclusiones se pueden extraer respecto al aprendizaje de la red. ¿Coinciden las regiones más relevantes con las que un humano consideraría importantes? ¿Existen zonas ignoradas por el modelo que podrían ser útiles?
   d) **Evaluación crítica:** Valorar si los mapas de activación proporcionan una explicación clara y coherente de las decisiones del modelo y discutir sus posibles limitaciones.

IMPORTANTE: La evaluación de este apartado no se centrará en la correcta implementación de GradCAM, sino en la **calidad del análisis crítico y científico**. Se espera que los estudiantes justifiquen sus observaciones, discutan los hallazgos con rigor y conecten los resultados con el comportamiento de la red y la teoría vista en el curso.

Entregable

Se deberá entregar una **memoria de un máximo de 7 páginas en formato pdf**, denominada **MemoriaCCN**, con fuente de tamaño 12pt, en la que se describan los experimentos realizados y las conclusiones derivadas de los mismos. Dentro de este límite deben incluirse las figuras clave, por lo que se recomienda centrarse en los aspectos técnicos y analíticos más relevantes, evitando introducir contenidos ampliamente conocidos (e.g., definición de una convolución, formulaciones matemáticas básicas, etc.). En caso necesario, podrán añadirse **Anexos** para incluir figuras, resultados o imágenes adicionales que complementen la discusión del texto principal.

Adicionalmente, será obligatorio entregar el **Notebook con el código fuente**. Para garantizar su claridad, cada bloque de código debe ir acompañado de una breve descripción que explique su propósito dentro del flujo de trabajo.

IMPORTANTE: Con el fin de asegurar una evaluación justa y coherente, es imprescindible que la memoria siga estrictamente el orden y los puntos establecidos en el Cuadro 1. Esto garantizará que todos los aspectos fundamentales de la práctica sean abordados de forma completa y sistemática. La estructura y el contenido de la memoria deberán reflejar cada uno de los apartados especificados, ofreciendo un análisis detallado, crítico y bien argumentado en cada sección.

Apartado 2: Despliegue del mejor modelo en un proveedor en la nube: AWS

Objetivos

Una vez entrenadas las redes neuronales del apartado anterior, es importante desplegarlas como una función invocable en la API y que puedan integrarse con aplicaciones y servicios. Esta funcionalidad está implementada en AWS, dentro del entorno AWS Sagemaker Studio. Por tanto, este segundo apartado consistirá en desplegar uno de esos modelos en dicho entorno, concretamente el que haya obtenido la mayor precisión (accuracy). Se va a emplear el modelo desarrollado en el apartado anterior para crear un endpoint en la nube, que permite hacer inferencia del modelo como una API que se puede invocar desde cualquier cliente HTTP. Para ello, se va a usar Amazon SageMaker AI y, en ese entorno, se desplegará el modelo como endpoint. En esta primera parte, el estudiante debe acceder al entorno de Amazon Sagemaker AI y crear un dominio y un usuario de dominio.

Antes de esto, el estudiante debe descargar el fichero del mejor modelo desarrollado en el apartado anterior. Para ello se usará el formato estándar de Keras (h5) y el formato asociado a TF Serving [6], que se usa para crear endpoints y es uno de los formatos adecuados para integrarlo en AWS Sagemaker AI. Para hacer esto, se debe modificar el código entregado en la primera parte solo para el modelo seleccionado y añadir las sentencias necesarias para guardar dicho modelo en los dos formatos. En el primer caso, el modelo debe llamarse **model.h5** y, en el segundo formato, dado que se almacena en un directorio, se debe comprimir en un archivo tar.gz: **model.tar.gz**.

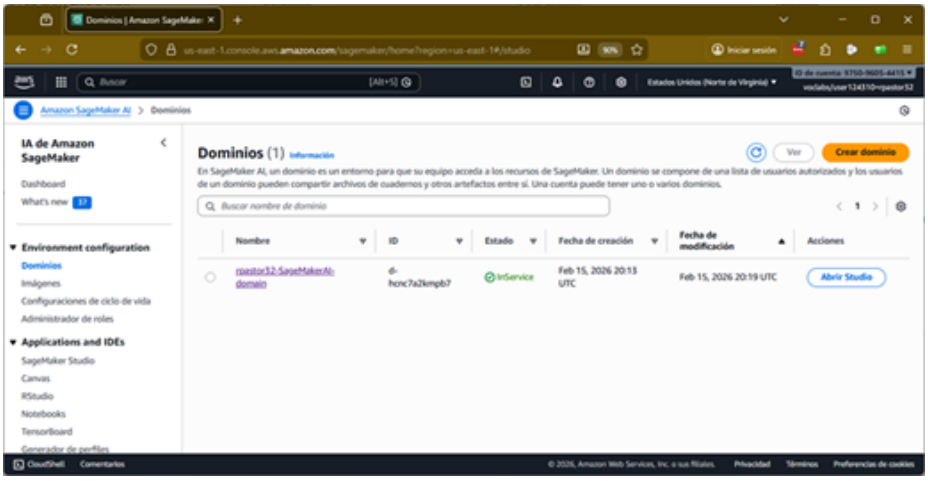


Figura 2: SageMaker AI Domain

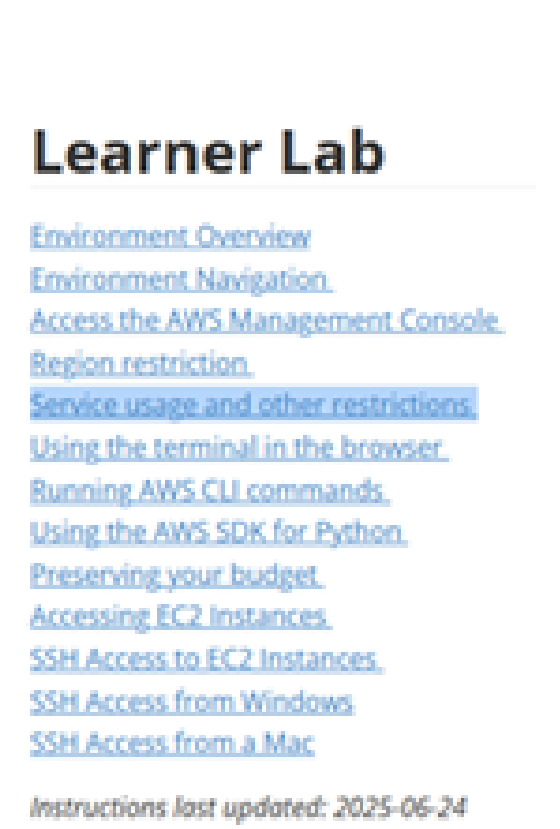
Creación del entorno de despliegue

Las instrucciones para crear ambos recursos están disponibles en la ayuda del entorno de laboratorio. Se recomienda leer toda la sección correspondiente a Amazon SageMaker. Para ello, en la consola de lanzamiento de AWS, use el enlace Service usage and other restrictions, como se muestra en la Figura 3a).

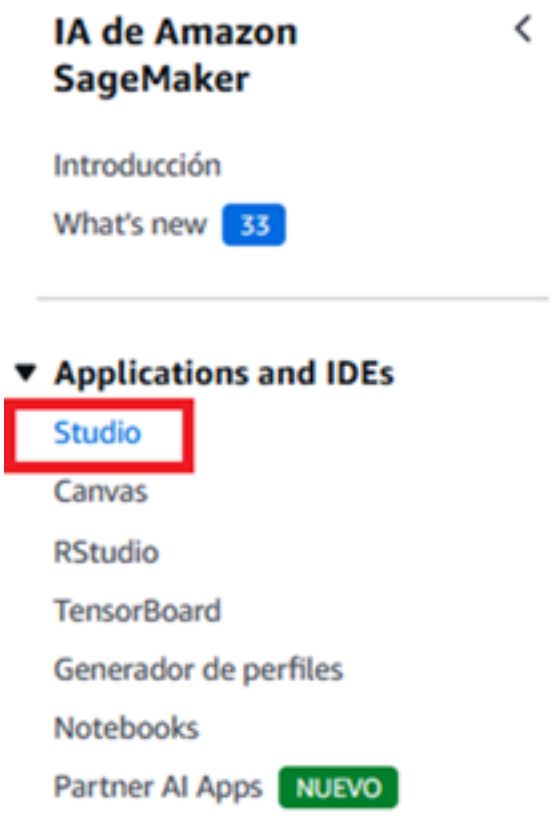
Una vez creado el dominio y el usuario, se puede acceder a la herramienta Studio, localizada en el panel lateral izquierdo, como se muestra en la Figura 3b. Este enlace abrirá la herramienta Studio, mostrada en la Figura 3c.

Para desplegar el modelo entrenado en el apartado anterior, se necesita ejecutar código desde la propia herramienta Studio, por lo que se empleará JupyterLab para crear un espacio de trabajo (JupyterLab) que permita crear notebooks y scripts con el código necesario para desplegar (deploy)

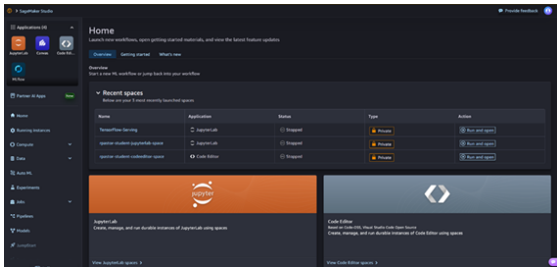
el modelo en un endpoint. Para ello se debe seleccionar la aplicación JupyterLab, mostrada en la Figura 3d.



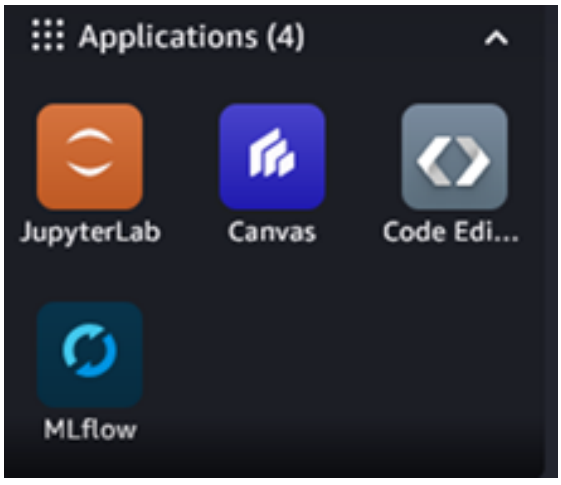
(a) SageMaker Domain Instructions



(b) Studio Link



(c) Herramienta Studio



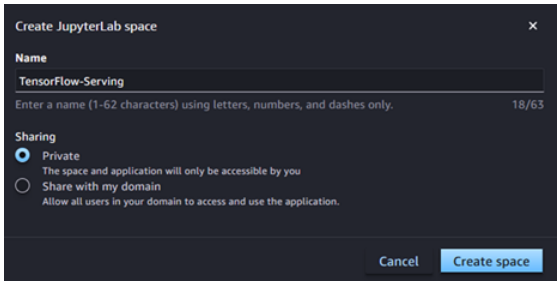
(d) Aplicaciones en Studio

Figura 3: Comparación entre los mejores modelos clásicos y neuronales en (a) regresión, (b) clasificación binaria y (c) clasificación multiclase. Las barras muestran la media a través de distintas semillas, y las barras de error representan la variabilidad. Cada conjunto de datos en el eje X se indica mediante el número de muestras (S) y características (F).

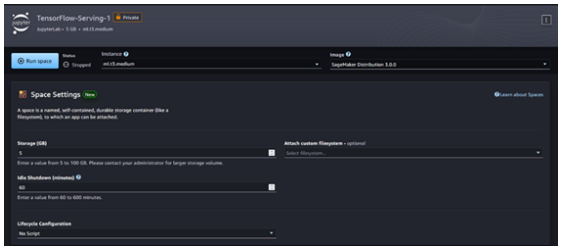
En la pantalla de espacios, hay que pulsar el botón <Create JupyterLab space>. Hay que indicar el nombre y la visibilidad (dejar como private) y pulsar Crear espacio, como se muestra en la Figura 4a y dejar en el siguiente paso los valores por defecto (ver Figura 4b).

Finalmente, se debe pulsar el botón <Run space>. Como se muestra en la Figura 4c, debe aparecer que se está ejecutando mediante el icono en verde.

Una vez que tenemos el espacio, ya solo queda abrirlo (acción Open) para acceder al entorno de despliegue que se va a utilizar para el modelo desarrollado, como se muestra en la Figura 4d.



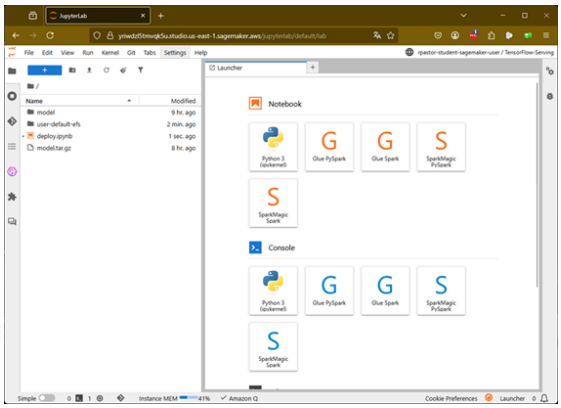
(a) Creación del espacio JupyterLab, paso 1



(b) Creación del espacio JupyterLab, paso 2



(c) Espacios de ejecución de JupyterLab



(d) JupyterLab ejecutándose en SageMaker Studio

Figura 4: Proceso de creación y ejecución de un entorno JupyterLab en SageMaker Studio: (a) y (b) configuración inicial del espacio; (c) gestión de espacios de ejecución; (d) entorno JupyterLab en funcionamiento.

Desarrollo del código para desplegar el modelo

Una vez que se cuenta con el espacio de JupyterLab disponible, se puede ejecutar el código necesario para desplegar el modelo. Para ello, se recomienda que el estudiante lea las referencias [3] y [4] e implemente el código necesario en un notebook llamado "deploy":

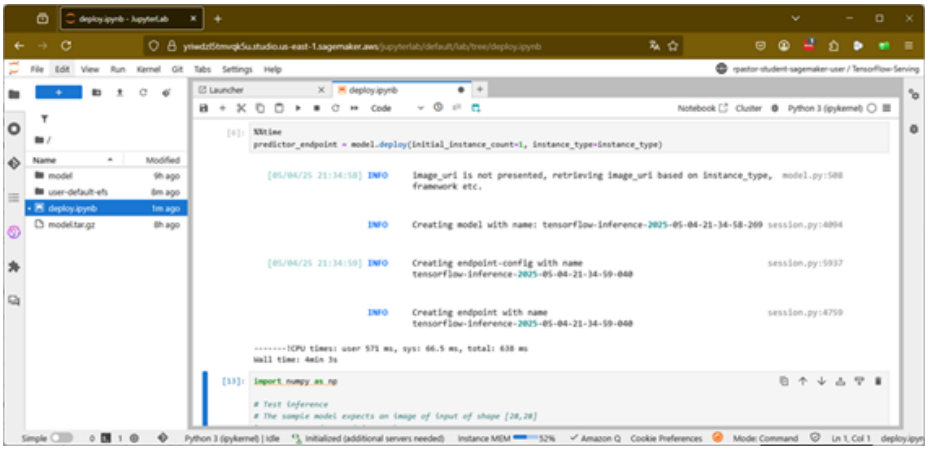
Una vez que el endpoint está desplegado desde el código del notebook (tardará un rato, del orden de 4/5 minutos), se puede trabajar con el endpoint, véase la Figura 5a.

También es posible visualizar su creación en la opción <Deployments>del menú lateral izquierdo, tanto mientras se está creando (ver Figura 5b) como una vez creado (ver Figura 5c).

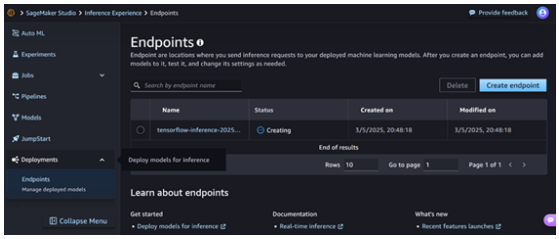
El notebook debe incorporar el código necesario para probar el endpoint y, al finalizar, es muy importante que se elimine el endpoint. Se puede hacer desde la UI o directamente desde el notebook:

```
predictor_endpoint.delete_endpoint()
```

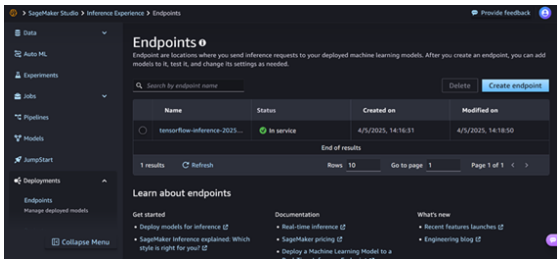
Nota: *Es muy importante que cuando no se use el lab, todos los recursos de Amazon Sagemaker AI Studio se paren, incluyendo la eliminación de los endpoint y parar el espacio de JupyterLab.*



(a) Despliegue del modelo



(b) Creación del endpoint



(c) Endpoint creado y disponible

Figura 5: Proceso de despliegue y publicación de un modelo en SageMaker: (a) despliegue inicial, (b) creación del endpoint y (c) endpoint operativo y disponible para inferencia.

Entregables

Se deberá entregar una **memoria de un máximo de 10 páginas en formato pdf**, denominada **MemoriaNube**, con fuente de tamaño 12pt, en la que **describan** en detalle las evidencias de despliegue del entorno (incluir: pantallazo de la ventana principal de la herramienta Studio, pantallazo de la ventana principal del espacio JupyterLab y pantallazo del endpoint desplegado desde la UI de Sagemaker Studio) y las evidencias de la inferencia del modelo a través del endpoint. Estas evidencias pueden estar en el notebook, pero, dado que se pueden desarrollar otros clientes externos, también pueden incorporarse en la memoria como el estudiante crea conveniente. En la memoria, en un apartado específico, deben describirse los problemas encontrados y las mejoras que se pueden considerar relativas al despliegue.

También se debe entregar el notebook ejecutado correctamente (deploy) y documentado correctamente, como se indica en el apartado anterior, y los dos ficheros binarios de los modelos (h5 y tar.gz).

Evaluación

La evaluación de esta práctica se basará exclusivamente en la memoria entregada, de acuerdo con la distribución de puntos recogida en el Cuadro 1. La nota final máxima será de 10 puntos, siendo necesario alcanzar al menos 5 para superar la actividad. Indicaciones importantes:

- No se valorarán definiciones generales ni formalizaciones matemáticas ampliamente conocidas. Lo que interesa es la **discusión técnica y crítica de los resultados obtenidos**, así como

Cuadro 1: Rúbrica o distribución de puntos para la evaluación.

Concepto	**Puntos**
1. Descripción y obtención de resultados de los diferentes modelos	3
1.1. Modelo 1: CNN (línea base)	*1*
1.2. Modelo 2: Transfer Learning	*1*
1.3. Modelo 3: Fine-Tuning	*1*
2. Explicabilidad con GradCAM	3
2.1. Visualización con GradCAM	*1*
2.2. Análisis crítico	*2*
3. Despliegue en la nube	4
3.1. Evidencias del entorno de ejecución correctas	*1*
3.2. Evidencias de despliegue correctas	*1*
3.3. Problemas encontrados presentes y sugerencias de mejora reales y constructivas	*0.5*
3.4. Código correctamente documentado y explicado en el notebook	*1*
3.5. Ficheros presentes con los modelos en los dos formatos pedidos	*0.5*

su conexión con los conceptos teóricos.

- Como se ha indicado, esta práctica consta de **dos apartados diferenciados**, y **cada uno de ellos requiere su propio conjunto de entregables**:

  - **En el Apartado 1 (CNN)** se debe entregar: (i) Un archivo PDF con la memoria técnica (`MemoriaCNN.pdf`); y (ii) Un notebook ejecutado y comentado.
  - **En el Apartado 2 (Despliegue en la nube)** se debe entregar: (i) Un archivo PDF con la memoria del despliegue (`MemoriaNube.pdf`); (ii) Un notebook con el código de despliegue (`deploy.ipynb`); y (iii) Los dos archivos del modelo en los formatos requeridos: `model.h5` y `model.tar.gz`.

  Recordad que **es imprescindible respetar esta estructura de entregables** para que la práctica pueda ser correctamente evaluada.

En resumen, la evaluación no premiará la mera implementación técnica, sino la **capacidad de razonar, justificar y analizar científicamente** los resultados obtenidos en la práctica.

Referencias

[1] Train your first neural network: basic classification | TensorFlow Core. https://www.tensorflow.org/tutorials/keras/classification?hl=es-419.

[2] Tutorial: Basic Classification. https://tensorflow.rstudio.com/tutorials/keras/classification.html.

[3] AWS. Deploying directly from model artifacts. https://sagemaker.readthedocs.io/en/v2/frameworks/tensorflow/deploying_tensorflow_serving.html#deploying-directly-from-model-artifacts.

[4] AWS. Tensorflow models using sagemaker. https://aws.amazon.com/es/blogs/machine-learning/deploy-trained-keras-or-tensorflow-models-using-amazon-sagemaker/.

[5] Keras. Home - Keras Documentation. https://keras.io/.

[6] Keras. Serving tensorflow models with tfserving. https://keras.io/examples/keras_recipes/tf_serving/.